

Sistema de Gestão de Encomendas: Análise, Modelação e Conceção de uma Solução Orientada a Objetos

David Soares¹, Mário Figueira¹ e Masembo Zola¹

¹ Instituto Superior de Engenharia do Porto, Departamento de Engenharia
Eletrotécnica,
Rua Dr. António Bernardino de Almeida 431, 4249-015 Porto, Portugal
{1251971, 1251824, 1251825}@isep.ipp.pt

Resumo. Este relatório descreve a análise, modelação, conceção e implementação de um sistema de gestão de encomendas orientado a objetos, desenvolvido em C++. O sistema permite registar clientes, criar encomendas entre depósitos, gerir veículos, acompanhar estados de entrega e consultar informação global da aplicação. A solução encontra-se organizada em camadas, separando a interface de utilizador, a coordenação da aplicação, os objetos de domínio, os repositórios, os serviços, os DTOs e os mappers. A classe Sistema atua como controlador principal, coordenando os fluxos de execução, enquanto as Views são responsáveis pela interação com o utilizador. Os dados são transportados entre as camadas através de DTOs, evitando que a interface aceda diretamente às entidades do domínio. Foram aplicados princípios de programação orientada a objetos, notação UML e padrões GRASP, com especial atenção à coesão, ao baixo acoplamento e à separação de responsabilidades.

Palavras-chave: Gestão de Encomendas · Engenharia de Software · C++ · UML · GRASP · Programação Orientada a Objetos · Sistemas de Informação.

1 Introdução

O envio de encomendas entre pessoas é uma necessidade do dia a dia, seja para fazer chegar um presente a um familiar, despachar documentos entre empresas ou simplesmente trocar objetos entre vizinhos. Em contextos urbanos de grande dimensão esta atividade encontra-se bem servida por operadores logísticos consolidados, mas em contextos locais — como uma pequena cidade com apenas alguns pontos de distribuição — a coordenação entre quem envia, quem recebe e os meios de transporte disponíveis é frequentemente feita de forma improvisada, recorrendo a telefonemas, registos em papel ou folhas de cálculo partilhadas. O sistema encontra-se atualmente implementado, desenvolvido com as principais funcionalidades definidas na fase de análise e desenho.

1.1 Objetivo e Âmbito

O objetivo deste projeto é desenvolver um sistema de gestão de encomendas, implementado em C++, que permita organizar o processo de registo, transporte e acompanhamento de encomendas entre diferentes pontos de distribuição. A aplicação pretende responder a um cenário local, onde é necessário controlar clientes, depósitos, veículos e encomendas de forma simples, estruturada e eficiente.

O sistema permite que um cliente se registe, crie encomendas, selecione o depósito de origem e o depósito de destino e acompanhe o estado da sua entrega. Do lado administrativo, a aplicação disponibiliza funcionalidades para adicionar e remover veículos, remover clientes, avançar o estado das encomendas e consultar informação global sobre os dados registados no sistema.

O âmbito do projeto centra-se numa aplicação local em modo consola, sem persistência permanente de dados. Assim, os dados são armazenados em memória durante a execução do programa, através de repositórios internos. Esta decisão permite focar o desenvolvimento na correta aplicação dos princípios de programação orientada a objetos, na organização do código e na separação de responsabilidades entre as diferentes classes.

A solução foi desenvolvida com base numa arquitetura em camadas, separando a interface de utilizador, a lógica de controlo, os objetos de domínio, os repositórios, os serviços, os DTOs e os mappers. Esta organização contribui para reduzir o acoplamento entre classes, aumentar a coesão do sistema e facilitar a manutenção futura do projeto.

Para além do objetivo funcional, o projeto tem também um objetivo pedagógico: aplicar conceitos de Engenharia de Software, modelação UML e princípios GRASP, demonstrando como uma solução orientada a objetos pode ser estruturada de forma modular, clara e evolutiva.

1.2 Atores e User Stories

Foram considerados dois atores principais: o Cliente, que utiliza o sistema para registrar-se, criar e acompanhar encomendas, e o Administrador, responsável pela gestão dos recursos e consulta global da aplicação.

Tabela 1–Tabela dos atores e user stories

US	Ator	Funcionalidade
US01	Cliente	Registrar-se no sistema para obter um identificador
US02	Cliente	Criar uma encomenda indicando cliente, origem, destino, descrição e peso.
US03	Cliente	Consultar o estado de uma encomenda associada ao próprio cliente.
US04	Cliente	Cancelar uma encomenda quando as regras do sistema o permitirem
US05	Administrador	Adicionar veículos ao sistema.
US06	Administrador	Remover veículos, desde que não estejam associados a entregas ativas.
US07	Administrador	Remover clientes registados, respeitando as regras de consistência.
US08	Administrador	Consultar informação global sobre clientes, depósitos, veículos e encomendas.

1.3 Requisitos Principais

Os requisitos do relatório foram compactados numa tabela única, separando funcionalidades, qualidades esperadas e restrições tecnológicas.

Tabela 2-Tabela de Requisitos

Categorias	Requisitos
Funcionais	Registrar cliente; criar encomenda; selecionar depósitos de origem e destino; atribuir ou reservar veículo; consultar estado; cancelar encomenda; adicionar/remover veículo; remover cliente; consultar informação global.
Não funcionais	Interface simples em consola; resposta rápida; consistência dos estados das encomendas; código modular; separação clara de responsabilidades; facilidade de manutenção
FURPS+	Usability: menus simples; Reliability: estados controlados; Performance: execução local rápida; Supportability: código por camadas; Constraints: C++, consola e armazenamento em memória.
Persistência	Nesta versão, os dados são mantidos apenas em memória durante a execução. A persistência em ficheiros ou base de dados fica como melhoria futura

1.4 Matriz de Rastreabilidade

A matriz foi reduzida para relacionar diretamente as funcionalidades principais com os casos de uso e com os módulos do código que as realizam.

Tabela 3-Matriz de Rastreabilidade

US	Caso de Uso	Módulos principais
US01	Registar Cliente	MenuView, ClienteView, ClienteInDto, Sistema, ClienteService, ClienteRepository, ClienteMapper.
US02	Criar Encomenda	MenuView, ClienteView, EncomendaInDto, Sistema, EncomendaService, EncomendaRepository, VeiculoService, EncomendaMapper.
US03	Consultar Estado da encomenda	ClienteView, Sistema, EncomendaService, EncomendaRepository, EncomendaOutDto.
US04	Cancelar Encomenda	ClienteView, Sistema, EncomendaService, EncomendaRepository, VeiculoRepository.
US05	Adicionar Veículo	AdministradorView, VeiculoInDto, Sistema, VeiculoService, VeiculoRepository, VeiculoMapper.
US06	Remover Veículo	AdministradorView, Sistema, VeiculoService, VeiculoRepository, EncomendaRepository.
US07	Remover Cliente	AdministradorView, Sistema, ClienteService, ClienteRepository.
US08	Consultar Informação Global	AdministradorView, Sistema, Service, Repositories, OutDTOs e Mappers.

1.5 Modelo de caso de uso

Neste capítulo é apresentada a estrutura de desenho do sistema de gestão de encomendas, recorrendo a diferentes modelos e diagramas UML que permitem representar a aplicação sob várias perspetivas. O objetivo desta secção é demonstrar como o sistema foi organizado internamente, quais são as principais entidades envolvidas, como estas se relacionam entre si e de que forma os diferentes componentes colaboram durante a execução das funcionalidades.

O design do sistema inclui o modelo de domínio, que representa os principais conceitos do problema, o diagrama de classes, que descreve a estrutura interna da aplicação, os diagramas de sequência, que mostram a interação entre os objetos durante os principais processos, e o diagrama de estados da encomenda, que representa o ciclo de vida de uma entrega. Para além disso, são apresentados os diagramas de pacotes, componentes e

instalação, permitindo compreender a organização lógica do código, a divisão dos módulos e a forma como a aplicação é executada localmente.

Através destes diagramas, procura-se evidenciar a separação de responsabilidades entre as diferentes camadas do sistema, nomeadamente View, Controller, DTOs, Mappers, Services, Repositories e Domain. Esta organização contribui para uma solução mais modular, com menor acoplamento entre classes e maior facilidade de manutenção e evolução futura.

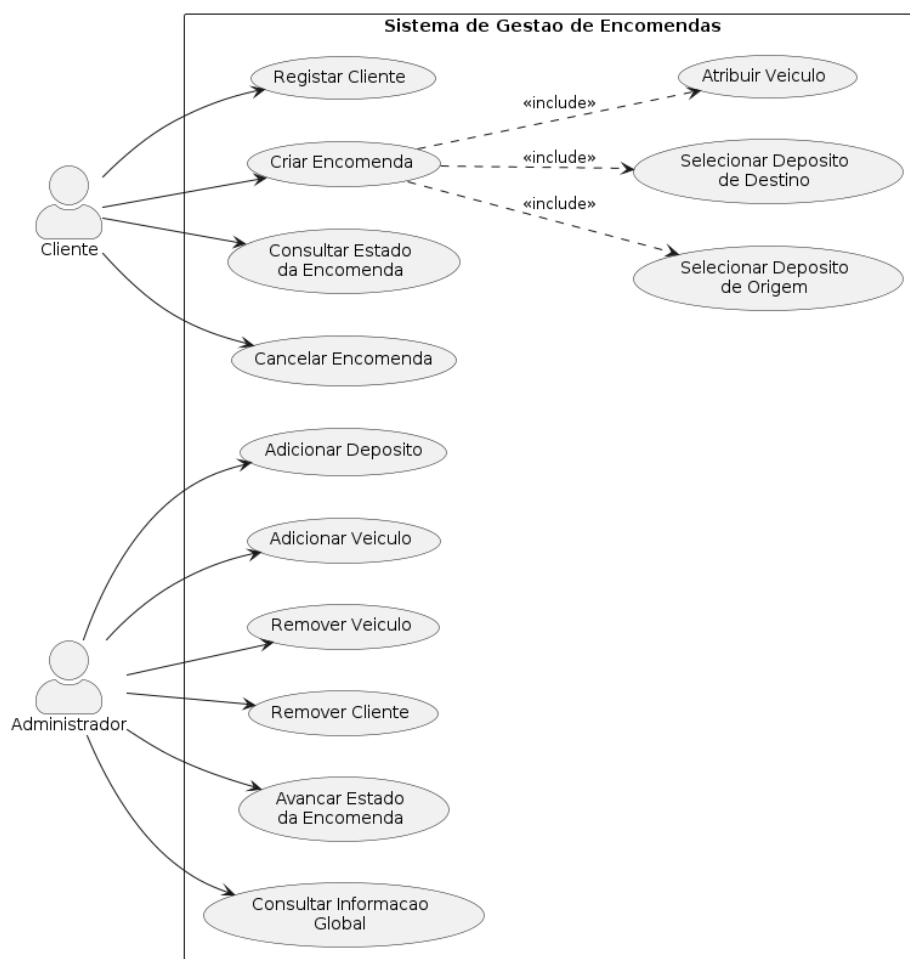


Fig 1-Diagrama do caso de uso

1.5.1 Especificações dos Casos de Usos

As especificações dos casos de uso apresentam, de forma detalhada, as principais funcionalidades disponibilizadas pelo sistema e a forma como os utilizadores interagem com elas. Para cada caso de uso são identificados o ator principal, a descrição da funcionalidade, as pré-condições necessárias, as pós-condições esperadas, o fluxo principal de execução, os fluxos alternativos e as regras de negócio associadas. Esta organização permite compreender

melhor o comportamento esperado do sistema antes e durante a implementação, garantindo que cada funcionalidade responde às necessidades identificadas na fase de análise.

1.5.1.1 UC 1 — Registar Cliente

A Tabela 4, mostra as especificações do caso de uso registar cliente.

Tabela 4- Especificações do caso de uso registar cliente.

Ator Principal	Cliente
Nome	Registar Cliente
Descrição	É o primeiro passo do sistema, permitindo a criação do usuário.
Pré-condições	O sistema é inclusivo para todos, sem registrações de idade
Pós-condições	Após o registo, o cliente é armazenado no sistema e recebe um número de identificação
Fluxo Principal	1. Utilizador faz cadastro 2. Sistema cria o cliente 3. Sistema gera automaticamente o número de identificação (ID) 4. Cliente fica registado 5. Sistema devolve o número de identificação (ID)
Fluxos Alternativos	2a. Nome inválido 2a1. Sistema rejeita o registo 2a2. Sistema informa erro 2b. Contacto inválido 2b1. Sistema rejeita registo 2c. Contacto Já existe 2c1. Sistema rejeita registo
Regras de Negócios	Só é possível fazer o cadastro uma vez.

1.5.1.2 UC 2 — Criar Encomenda

A Tabela 5, mostra as especificações do caso de uso Criar Encomenda

Tabela 5- Especificações do caso de uso Criar Encomenda.

Ator Principal	Cliente
Nome	Criar Encomenda

Descrição	Permite ao cliente criar uma nova encomenda no sistema, definindo o depósito de origem e o depósito de destino do transporte.
Pré-condições	O cliente deverá encontrar-se registado no sistema. Deverão existir, pelo menos, dois depósitos disponíveis.
Pós-condições	A encomenda fica registada no sistema. Um veículo disponível é associado automaticamente à encomenda. O estado inicial da encomenda é definido.
Fluxo Principal	1. O cliente inicia o processo de criação de encomenda. 2. O sistema solicita os dados da encomenda. 3. O cliente seleciona o depósito de origem. 4. O cliente seleciona o depósito de destino. 5. O sistema valida e regista a encomenda. 6. O sistema atribui automaticamente um veículo disponível.
Fluxos Alternativos	5a. Dados da encomenda inválidos ou incompletos: o sistema solicita a correção das informações antes de prosseguir. 6a. Não existem veículos disponíveis: o sistema regista a encomenda com estado pendente e informa o cliente dessa situação.
Regras de Negócio	O depósito de origem e o depósito de destino têm de ser distintos. O peso da encomenda tem de ser superior a zero.

1.5.1.3 UC 3 —Consultar Estado da Enomenda

A Tabela 6, mostra as especificações do caso de Consultar Estado da Encomenda

Tabela 6- Especificações do caso de uso Consultar Estado da Encomenda.

Ator Principal	Cliente
Nome	Consultar Estado da Encomenda
Descrição	Permite ao cliente acompanhar o estado atual de uma encomenda sua, desde a recolha no depósito de origem até à entrega no destino.
Pré-condições	O cliente deverá encontrar-se registado no sistema. A encomenda a consultar deverá estar associada a esse cliente.
Pós-condições	O cliente obtém a informação atualizada sobre o estado da encomenda.

Fluxo Principal	1. O cliente solicita a consulta do estado de uma encomenda. 2. O sistema solicita a identificação da encomenda. 3. O cliente fornece o identificador da encomenda. 4. O sistema apresenta o estado atual da encomenda.
Fluxos Alternativos	3a. Identificador inválido ou inexistente: o sistema informa o cliente e solicita nova introdução.
Regras de Negócio	Só é possível consultar encomendas associadas ao próprio cliente.

1.5.1.4 UC 4 — Cancelar Encomenda

A Tabela 7, mostra as especificações do caso de uso Cancelar Encomenda.

Tabela 7- Especificações do caso de uso Cancelar Encomenda

Campo	Descrição
Ator Principal	Cliente
Nome	Cancelar Encomenda
Descrição	Permite ao cliente cancelar uma encomenda previamente criada, desde que esta ainda não tenha sido entregue.
Pré-condições	O cliente deve estar registado no sistema. A encomenda deve existir no sistema. A encomenda deve estar associada ao cliente. A encomenda não pode estar no estado ENTREGUE nem CANCELADA .
Pós-condições	A encomenda passa para o estado CANCELADA . Caso exista um veículo associado à encomenda, este poderá ser libertado e voltar a estar disponível.
Fluxo Principal	1. O cliente seleciona a opção “Cancelar Encomenda”. 2. O sistema solicita o identificador da encomenda. 3. O cliente introduz o ID da encomenda. 4. O sistema procura a encomenda. 5. O sistema verifica se a encomenda pode ser cancelada. 6. O sistema altera o estado da encomenda para CANCELADA . 7. O sistema informa que a encomenda foi cancelada com sucesso.
Fluxos Alternativos	4a. A encomenda não existe: o sistema informa que não foi possível cancelar a encomenda. 5a. A encomenda já foi entregue: o sistema impede o cancelamento. 5b. A encomenda já está cancelada: o sistema informa que a operação não pode ser realizada novamente.
Regras de Negócio	Apenas encomendas existentes podem ser canceladas. Uma encomenda entregue não pode ser cancelada. Uma encomenda já cancelada não pode ser cancelada novamente. O cancelamento altera o estado da encomenda para CANCELADA .

1.5.1.4 UC 5 — Adicionar Veículo

A Tabela 8, mostra as especificações do caso de uso Adicionar Veículo

Tabela 8- Especificações do caso de uso Adicionar Veículo

Ator Principal	Administrador
Nome	Adicionar Veículo
Descrição	Permite ao administrador adicionar um novo veículo ao sistema, definindo as suas principais características. Após o registo, o veículo fica disponível para o transporte de encomendas.
Pré-condições	O administrador deverá estar autenticado no sistema.
Pós-condições	O veículo fica registado no sistema. O veículo fica disponível para ser atribuído a encomendas futuras.
Fluxo Principal	1. O administrador seleciona a opção "Adicionar Veículo". 2. O sistema solicita os dados do veículo. 3. O administrador introduz a matrícula, a capacidade máxima e os restantes atributos. 4. O sistema valida os dados introduzidos. 5. O sistema regista o veículo. 6. O sistema define o estado inicial do veículo como "disponível". 7. O sistema confirma ao administrador a adição do veículo.
Fluxos Alternativos	4a. Dados inválidos ou incompletos: o sistema solicita a correção antes de prosseguir com o registo.
Regras de Negócio	A matrícula do veículo tem de ser única no sistema. A capacidade máxima tem de ser superior a zero.

1.5.1.5 UC 6 — Remover Veículo

A Tabela 9, mostra as especificações do caso de uso Remover Veículo

Tabela 9- Especificações do caso de uso Remover Veículo

Ator principal	Administrador
Nome	Remover veículos

Descrição	Permite ao administrador remover um veículo da frota do sistema.
Pré-condições	O administrador deve estar autenticado no sistema. O veículo deve existir.
Pós-condições	O veículo é removido da frota. O veículo deixa de estar disponível no sistema.
Fluxo Principal	1.O administrador seleciona a opção Remover Veículo. 2.O sistema solicita o ID do veículo. 3.O administrador introduz o ID. 4.O sistema procura o veículo. 5.O sistema remove o veículo. 6.O sistema informa que o veículo foi removido com sucesso
Fluxo alternativo	4a. Veículo inexistente 4a1. O sistema não encontra o veículo. 4a2. O sistema informa que o veículo não existe. 5a. Veículo associado a entrega ativa 5a1. O sistema detecta que o veículo está associado a uma encomenda em transporte. 5a2. O sistema impede a remoção do veículo.
Regras de Negócio	Cada veículo possui um ID único. Apenas veículos registados podem ser removidos. Um veículo associado a uma entrega ativa não pode ser removido. Apenas o administrador pode remover veículos

1.5.1.6 UC 7 — Remover Cliente

A Tabela 10, mostra as especificações do caso de uso Remover Cliente.

Tabela 10-Especificações do caso de uso Remover Cliente

Ator Principal	Administrador
Nome	Remover Cliente
Descrição	Permite ao administrador remover um cliente registrado do sistema.
Pré-condições	O administrador deve estar autenticado no sistema O cliente deve existir.
Pós-Condições	O cliente é removido do sistema. O cliente deixa de constar na lista de clientes registrados.
Fluxo Principal	O administrador seleciona a opção Remover Cliente. O sistema solicita o ID do cliente. O administrador introduz o ID. O sistema procura o cliente. O sistema remove o cliente. O sistema informa que o cliente foi removido com sucesso
Fluxo Alternativo	4a. Cliente inexistente 4a1. O sistema não encontra o cliente. 4a2. O sistema informa que o cliente não existe.
Regra de Negócio	Cada cliente possui um ID único. Apenas clientes registrados podem ser removidos. Apenas o administrador pode remover clientes.

1.5.1.7 UC 8 — Informação Global

A Tabela 11, mostra as especificações do caso de uso Informação Global.

Tabela 11- Especificações do caso de uso Informação Global.

Ator Principal	Administrador
Nome	Consultar Informação Global
Descrição	Permite ao administrador consultar, de forma agregada, toda a informação do sistema — clientes, encomendas, veículos e depósitos — de modo a supervisionar o seu funcionamento.
Pré-condições	O administrador deverá estar autenticado no sistema.
Pós-condições	Nenhuma alteração aos dados do sistema.
Fluxo Principal	1. O administrador solicita a consulta global. 2. O sistema apresenta a lista de clientes registados. 3. O sistema apresenta a lista de encomendas, com os veículos atribuídos e os depósitos associados. 4. O sistema apresenta a lista de veículos e o seu estado atual. 5. O sistema apresenta a lista de depósitos e a respetiva localização.
Fluxos Alternativos	2a. Não existem registos a apresentar: o sistema informa o administrador dessa situação.
Regras de Negócio	A consulta é estritamente de leitura, não podendo originar modificações nos dados.

2. Desenho do Sistema

Neste capítulo é apresentada a estrutura de desenho do sistema de gestão de encomendas, recorrendo a diferentes modelos e diagramas UML que permitem representar a aplicação sob várias perspetivas. O objetivo desta secção é demonstrar como o sistema foi organizado internamente, quais são as principais entidades envolvidas, como estas se relacionam entre si e de que forma os diferentes componentes colaboram durante a execução das funcionalidades.

O design do sistema inclui o modelo de domínio, que representa os principais conceitos do problema, o diagrama de classes, que descreve a estrutura interna da aplicação, os diagramas de sequência, que mostram a interação entre os objetos durante os principais processos, e o diagrama de estados da encomenda, que representa o ciclo de vida de uma entrega. Para além disso, são apresentados os diagramas de pacotes, componentes e instalação, permitindo compreender a organização lógica do código, a divisão dos módulos e a forma como a aplicação é executada localmente.

Através destes diagramas, procura-se evidenciar a separação de responsabilidades entre as diferentes camadas do sistema, nomeadamente View, Controller, DTOs, Mappers, Services,

Repositories e Domain. Esta organização contribui para uma solução mais modular, com menor acoplamento entre classes e maior facilidade de manutenção e evolução futura.

2.1 Modelo de Domínio

O modelo de domínio (Domain Model) é uma representação visual dos principais conceitos do mundo real que o sistema pretende modelar, juntamente com as relações estáticas entre eles. Não se trata de um diagrama técnico sobre como o software vai ser construído, mas sim de uma forma de capturar, numa só imagem, o vocabulário do negócio — neste caso, o vocabulário da logística de encomendas. É por isso uma ferramenta particularmente útil para alinhar a equipa de desenvolvimento com os utilizadores e com eventuais clientes do sistema, ainda antes de se começar a programar.

No contexto deste projeto, o modelo de domínio reflete a realidade de uma pequena cidade com seis pontos de distribuição, onde clientes enviam encomendas entre si e veículos fazem o transporte entre esses pontos. Pretende-se, com este modelo, fornecer uma visão conceptual do problema, identificando as entidades fundamentais envolvidas no envio, transporte e entrega de encomendas, sem qualquer referência a pormenores de implementação.

2.1.1 Conceitos de Domínio

Foram identificados quatro conceitos principais: Cliente, Encomenda, Veículo e Depósito. O Cliente representa o utilizador do sistema responsável pela criação de encomendas. A Encomenda corresponde ao pedido de transporte efetuado por um cliente, contendo as informações essenciais para que o processo de entrega possa ser conduzido e acompanhado. O Veículo representa o meio de transporte utilizado para deslocar as encomendas entre os diferentes pontos da cidade. Finalmente, o Depósito corresponde a um ponto físico de recolha ou entrega, existindo vários desses pontos distribuídos pela área de operação do sistema.

2.1.2 Relações e Associações

Entre estes conceitos estabelecem-se várias associações. Cada Cliente pode criar encomendas no sistema, existindo por isso uma associação direta entre Cliente e Encomenda. Cada Encomenda está associada a dois Depósitos distintos — o de origem e o de destino — o que permite modelar explicitamente os pontos físicos envolvidos no processo de entrega. Adicionalmente, cada Veículo encontra-se associado às encomendas que transporta, sendo responsável pela deslocação destas entre o depósito de origem e o depósito de destino. No seu conjunto, estas associações permitem representar de forma clara o fluxo logístico do sistema.

2.1.3 Multiplicidades

No que se refere às multiplicidades, cada Cliente pode criar zero ou mais Encomendas, ao passo que cada Encomenda está associada a exatamente um cliente. Cada encomenda possui obrigatoriamente um Depósito de origem e um Depósito de destino, garantindo que os locais de recolha e de entrega estão sempre definidos. Um Veículo pode transportar várias encomendas ao longo do tempo, mas cada encomenda é transportada por um único veículo durante o seu ciclo de entrega.

2.1.4 Atributos

Cada entidade do modelo de domínio é caracterizada por um conjunto de atributos relevantes. O Cliente possui um identificador único, um nome e um contacto. A Encomenda é caracterizada por um identificador, uma descrição, um peso, um estado, um identificador do cliente, um identificador do depósito de origem, um identificador do depósito final e um do seu respetivo veículo, permitindo acompanhar o seu progresso ao longo do processo de

entrega. O Veículo inclui atributos como um identificador, matrícula, capacidade máxima e estado de disponibilidade, ao passo que o Depósito é descrito por um identificador, um nome, uma localização e uma capacidade máxima de armazenamento.

2.1.5 Diagrama do Modelo de Domínio

A Figura 2 apresenta o diagrama do modelo de domínio, sintetizando graficamente os conceitos, os atributos, as associações e as multiplicidades descritos nas subsecções anteriores. processo de entrega. Adicionalmente, cada Veículo encontra-se associado às encomendas que transporta, sendo responsável pela deslocação destas entre o depósito de origem e o depósito de destino. No seu conjunto, estas associações permitem representar de forma clara o fluxo logístico do sistema.

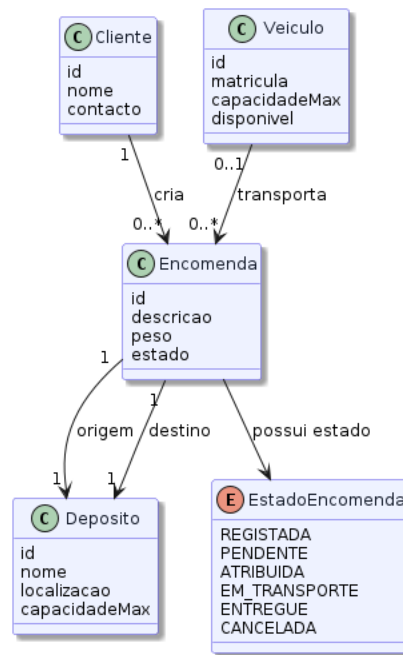


Figura 2 - Modelo de domínio do sistema de gestão de encomendas, evidenciando as entidades Cliente, Encomenda, Veículo e Depósito e as respetivas associações e multiplicidades

2.2 Diagrama de Classes

O diagrama de classes apresenta a estrutura interna do sistema, evidenciando as principais classes, os seus atributos, operações e relações existentes entre elas. Diferentemente do modelo de domínio, que se mantém num plano conceptual, o diagrama de classes já representa decisões de desenho do software e a organização interna da solução.

Como se pode observar na Fig. 3, a classe Sistema atua como controlador principal, centralizando a coordenação das funcionalidades da aplicação. Com a evolução da arquitetura, foram introduzidas classes Repository específicas para cada entidade do domínio — ClienteRepository, EncomendaRepository, VeiculoRepository e DepositoRepository — responsáveis pela gestão e armazenamento dos dados do sistema.

Esta abordagem permitiu reduzir o acoplamento entre classes, aumentar a coesão do sistema e aplicar de forma mais clara princípios GRASP, especialmente os padrões Controller e Low Coupling, contribuindo para uma solução mais modular, reutilizável e de fácil manutenção

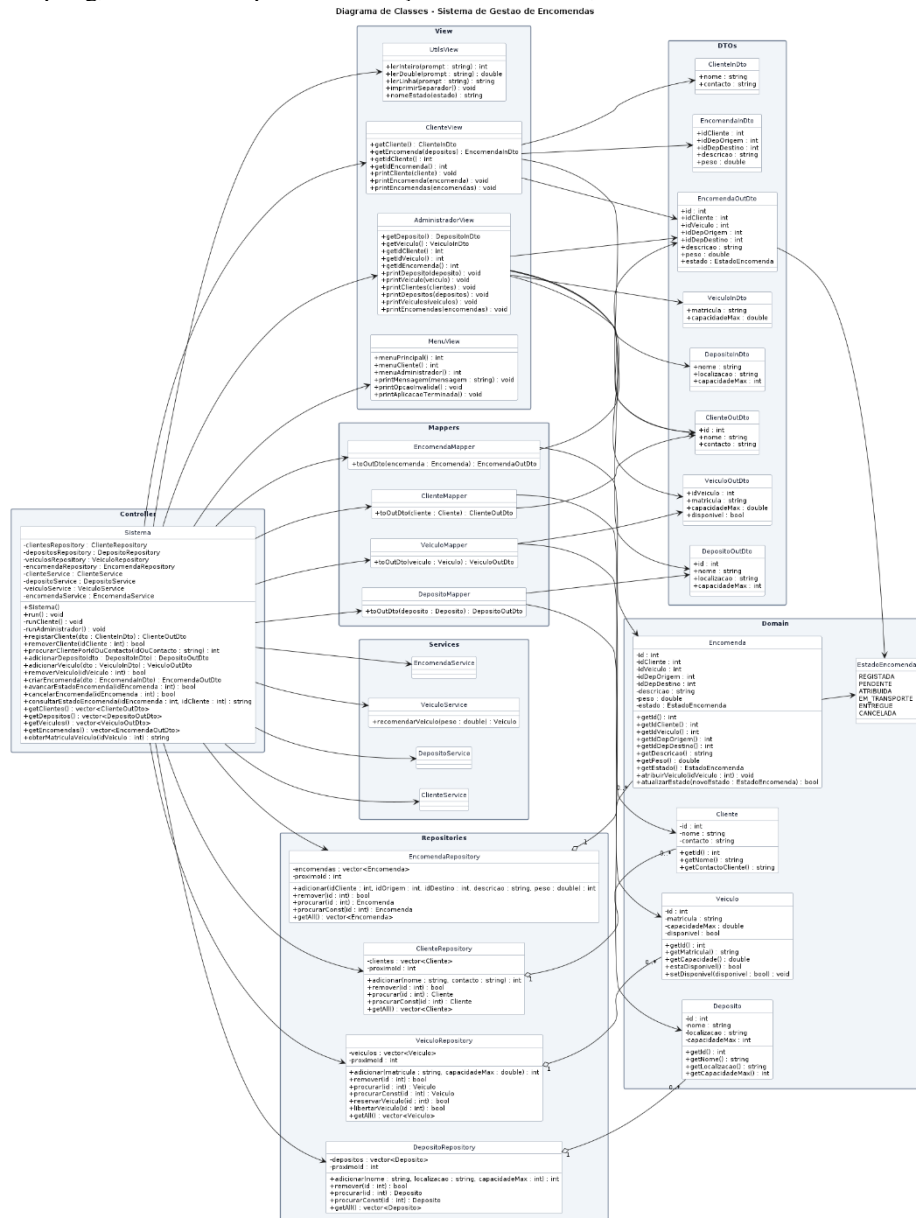


Fig. 3. Diagrama de classes do sistema de gestão de encomendas.

2.3 Diagramas de Sequência

Um diagrama de sequência de sistema (System Sequence Diagram, ou SSD) é um tipo específico de diagrama UML que mostra, ao longo do tempo, as mensagens trocadas entre um ator externo e o sistema visto como uma "caixa preta". O objetivo não é detalhar o funcionamento interno do software, mas sim evidenciar, de forma clara e ordenada, as interações que ocorrem em cada cenário: quem pede o quê, em que momento, e que resposta obtém do sistema. Os SSDs são, por isso, uma ponte natural entre os casos de uso — que descrevem o que acontece em texto corrido — e o desenho interno do sistema, que virá mais à frente.

Nesta secção apresentam-se os SSDs correspondentes aos sete casos de uso anteriormente detalhados, ilustrando a sequência de interações previstas entre cada ator e o sistema.

2.3.1 SSD: UC 1 — Registar Cliente

A Fig. 4 representa a interação entre o cliente e o sistema durante o processo de registo de um novo cliente. O utilizador inicia o processo através da interface principal, introduzindo os dados necessários, como nome e contacto. Em seguida, o sistema valida as informações fornecidas e verifica se o contacto já se encontra registado. Caso os dados sejam válidos e únicos, o cliente é criado e armazenado no repositório, sendo apresentada uma mensagem de confirmação. Caso contrário, o sistema informa que os dados são inválidos ou que o contacto já existe

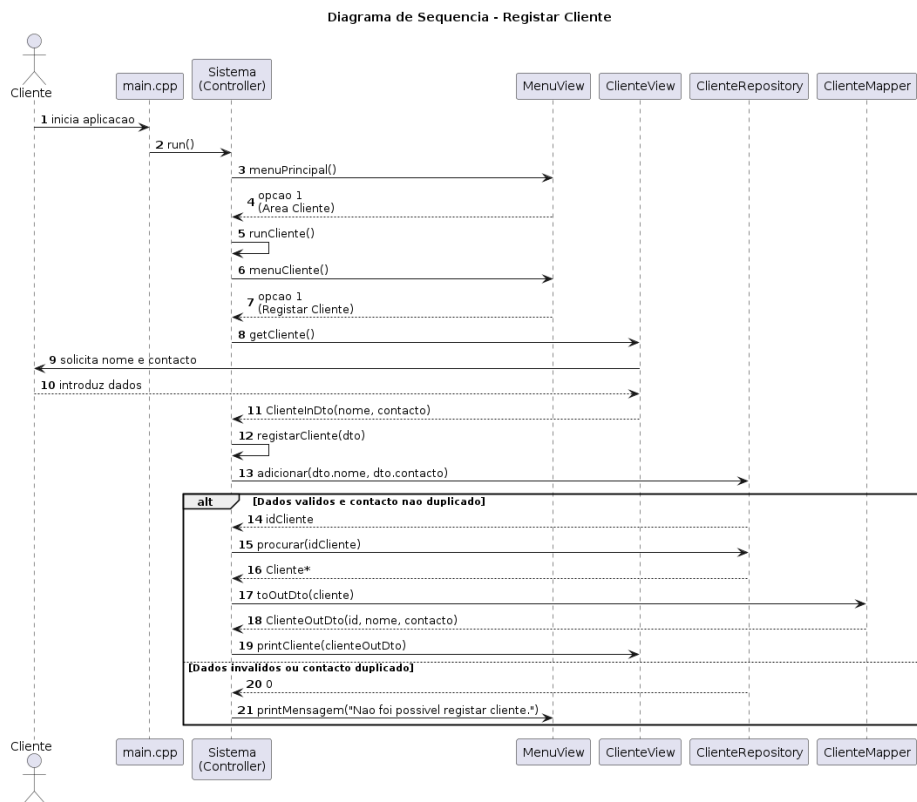


Fig. 4.SSD Registar Cliente

2.3.2 SSD: UC 2 — Criar Encomenda

A Fig. 5 representa a interação entre o cliente e o sistema durante o processo de criação de uma encomenda. O cliente inicia o processo, fornece os dados necessários e seleciona os depósitos de origem e de destino, sendo posteriormente notificado da confirmação da encomenda.

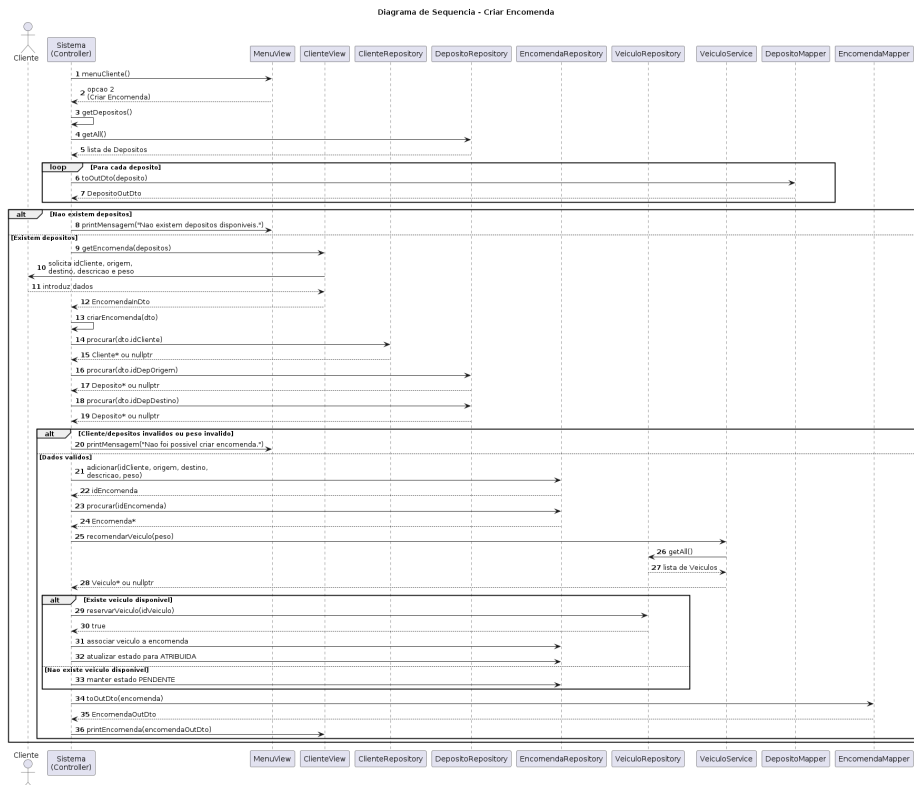


Fig 5. SSD Criar Encomenda

2.3.3 SSD: UC 3 — Consultar Estado da Encomenda

A Fig. 6 ilustra a sequência de eventos envolvidos na consulta do estado de uma encomenda. O cliente fornece a identificação da encomenda e o sistema devolve a informação atualizada sobre o estado em que esta se encontra

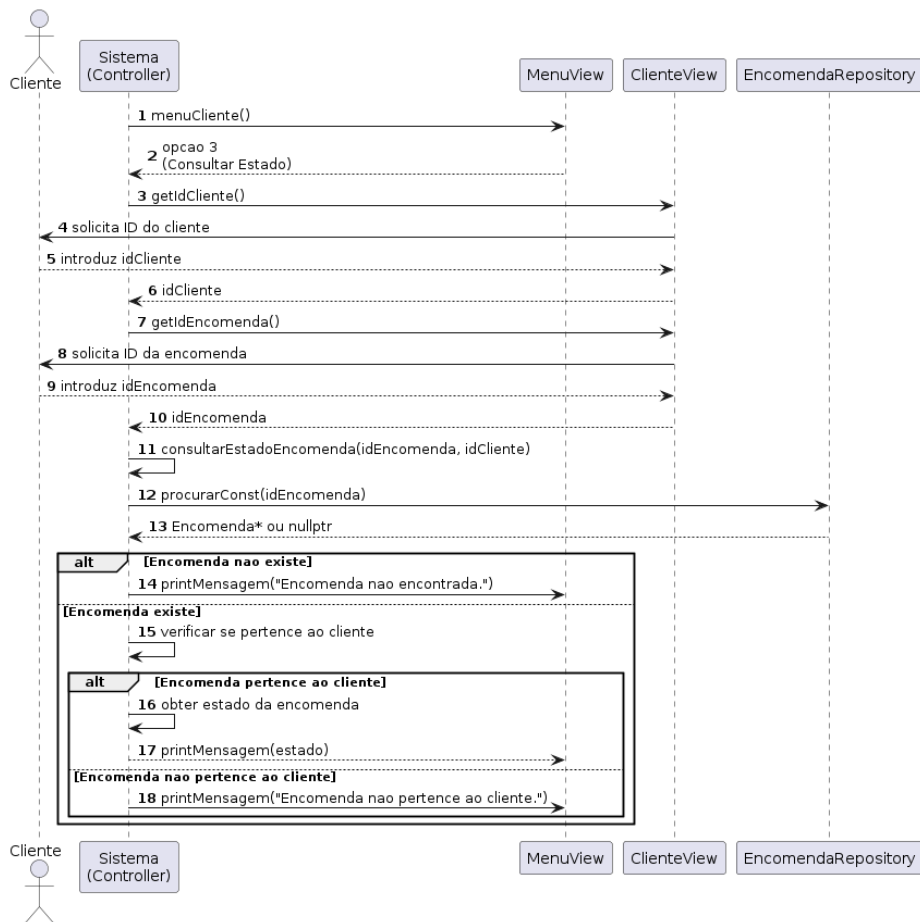


Fig. 6 SSD Consultar Estado da Encomenda

2.3.4 SSD: UC 4 — Cancelar Encomenda

O caso de uso Cancelar Encomenda permite ao cliente interromper uma encomenda previamente criada, desde que esta ainda não tenha sido entregue ou cancelada. O sistema procura a encomenda pelo seu identificador, verifica se o cancelamento é permitido e, em caso de sucesso, altera o seu estado para CANCELADA. Caso a encomenda tenha um veículo associado, este pode ser libertado para voltar a ficar disponível no sistema.

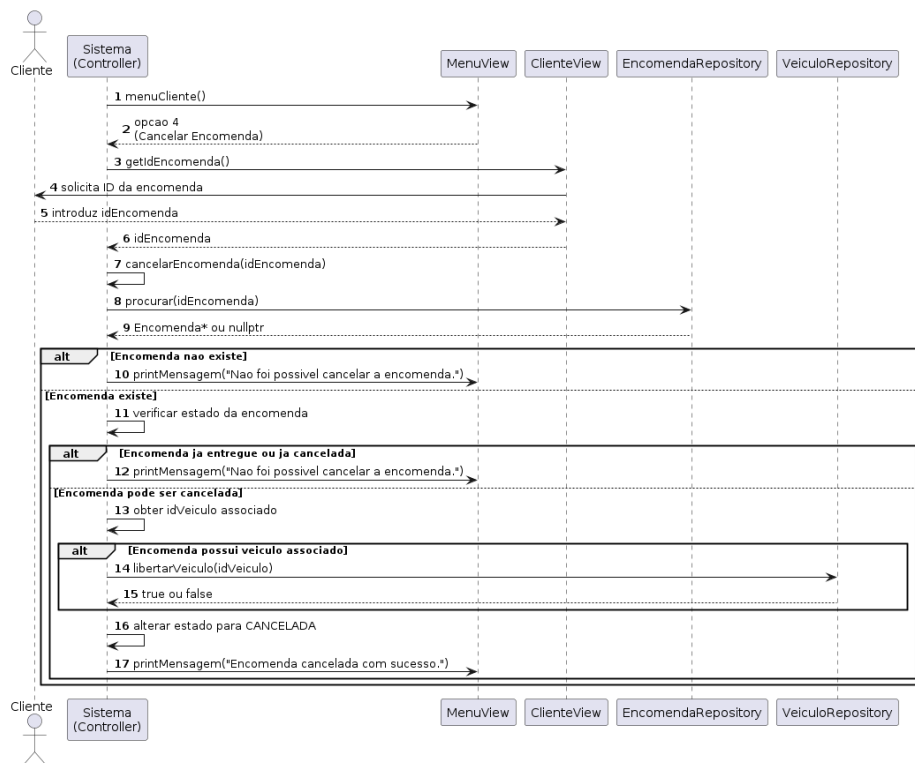


Fig 7. SSD Cancelar Encomenda

2.3.4 SSD: UC 5 — Adicionar Veículo

O diagrama de sequência do caso de uso “Adicionar Veículo” ilustra o processo de registo de um novo veículo por parte do administrador. O administrador introduz os dados do veículo, como matrícula e capacidade máxima, sendo estes validados pelo sistema. O sistema verifica ainda a existência de veículos com matrícula duplicada, garantindo a unicidade dos registos. Em caso de sucesso, o veículo é registado e disponibilizado para utilização; caso contrário, o sistema informa o administrador sobre a impossibilidade de realizar a operação.

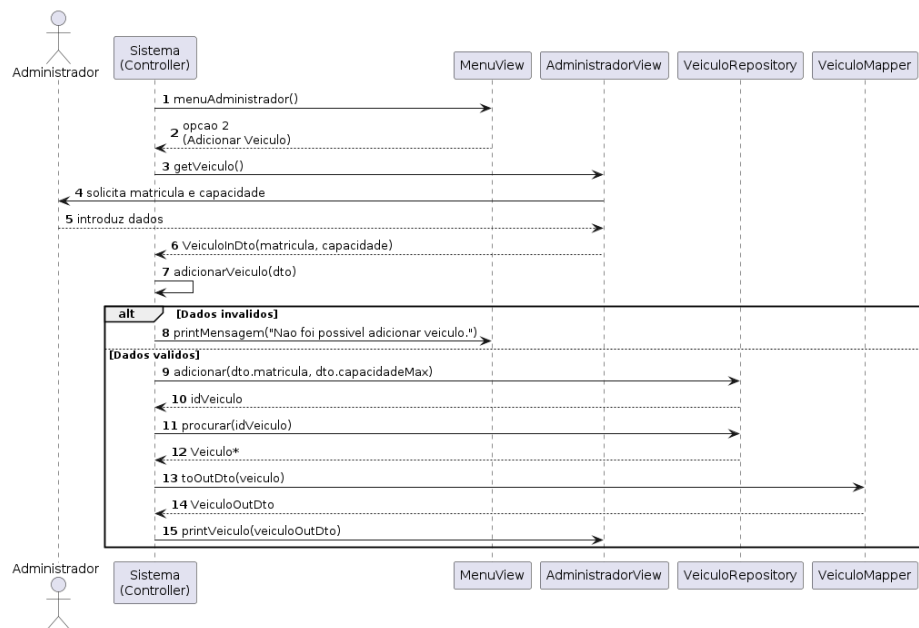


Fig 8. SSD Adicionar Veículo

2.3.5 SSD: UC 6 — Remover Veículo

O diagrama de sequência (Fig.9) associado ao caso de uso “Remover Veículo” descreve a sequência de interações necessárias para a remoção de um veículo do sistema. O administrador solicita a remoção de um veículo identificado pelo seu identificador, sendo esta operação validada pelo sistema. O sistema verifica se o veículo se encontra associado a encomendas ativas, impedindo a sua remoção em caso afirmativo. Caso não existam restrições, o veículo é removido com sucesso e o sistema informa o administrador do resultado da operação

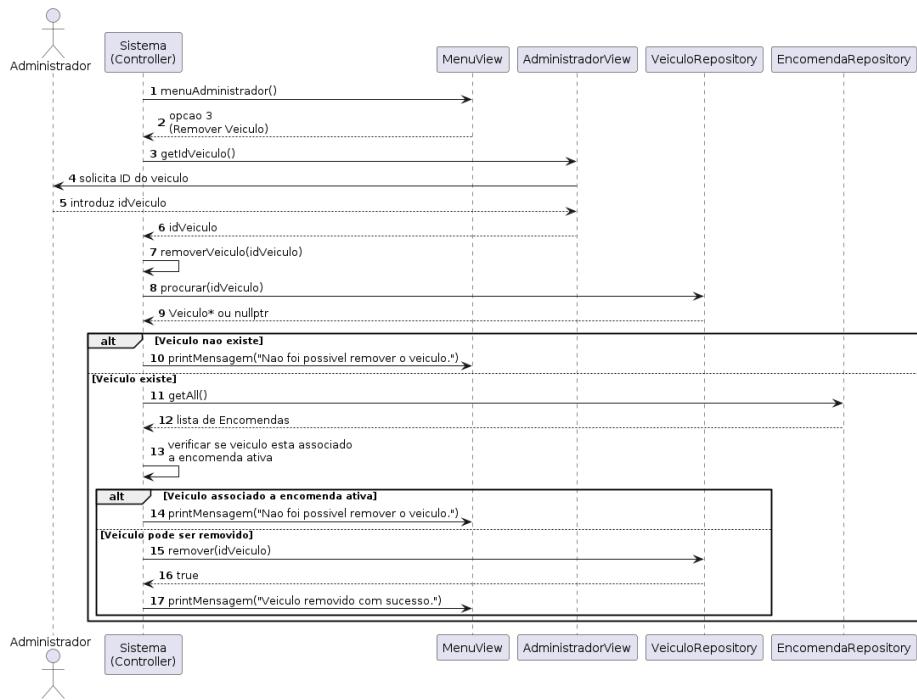


Fig 9. SSD Remover Veículo

2.3.6 SSD: UC 7 — Remover Cliente

O diagrama de sequência do caso de uso “Remover Cliente” (Fig.10) representa o processo de eliminação de um cliente do sistema por parte do administrador. Após a identificação do cliente, o sistema verifica a existência de encomendas associadas que ainda se encontrem ativas. Se tais encomendas existirem, a operação é recusada, garantindo a consistência dos

dados do sistema. Caso contrário, o cliente é removido e o sistema confirma a operação ao administrador.

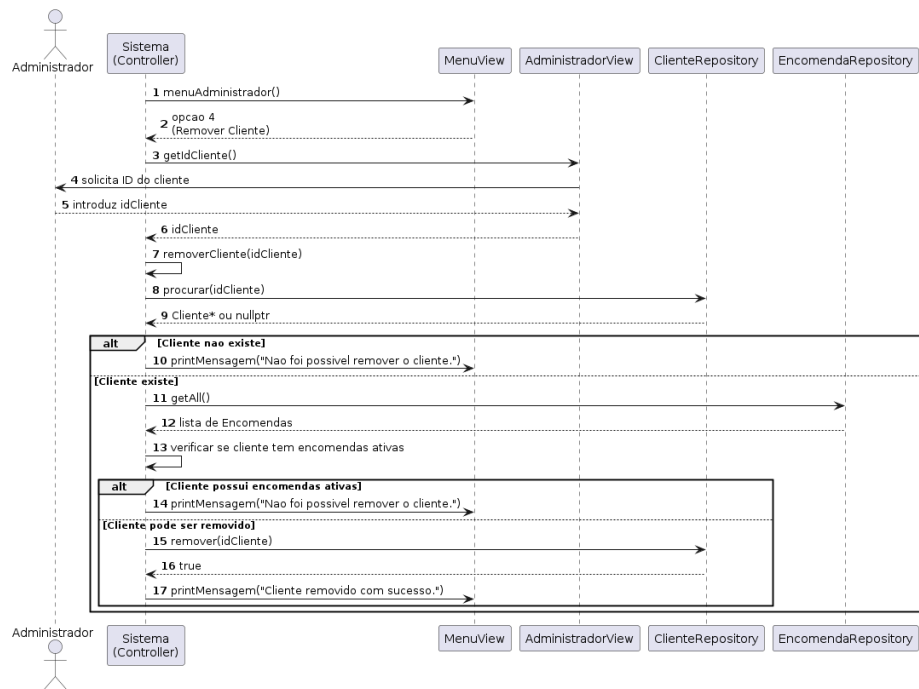


Fig 10. SSD Remover Cliente

2.3.7 SSD: UC 8 — Informação Global

O diagrama de sequência do caso de uso “Consultar Informação Global” (Fig. 11) ilustra a interação entre o administrador e o sistema para obtenção de uma visão agregada de todos os dados existentes. Representa a interação entre o administrador e o sistema na consulta de informação global. O sistema responde apresentando os dados relativos a clientes, encomendas, veículos e depósitos, de modo a permitir a supervisão do funcionamento geral da plataforma.

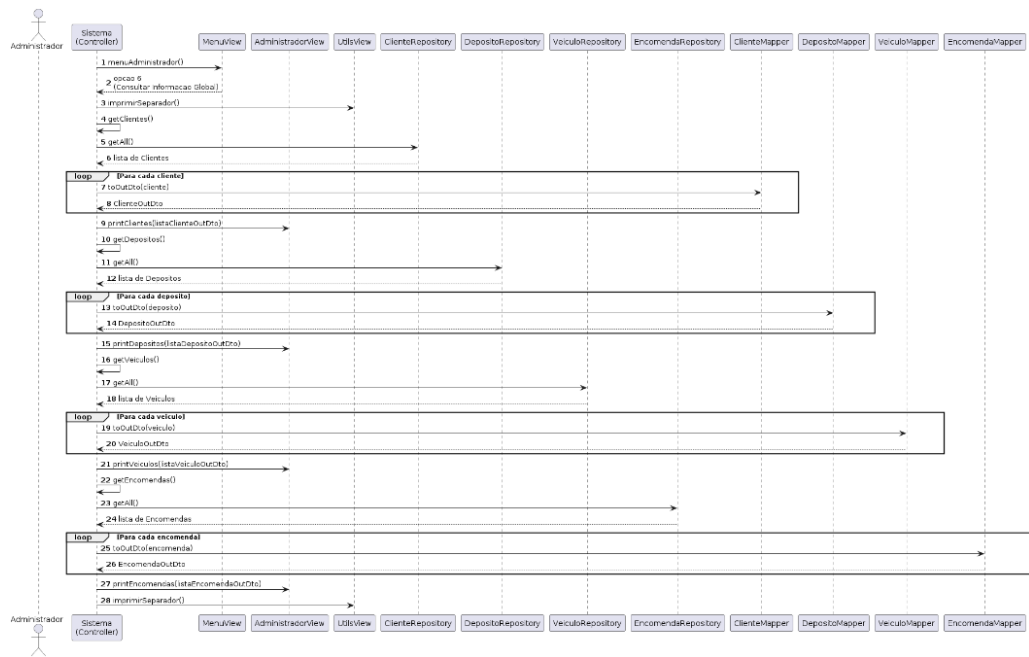


Fig 11. SSD Consultar Informação Global

2.4 Diagrama de estados

O atributo estado da Encomenda, apresentado no modelo de domínio (secção 5), é central para o acompanhamento do ciclo de vida de cada pedido. Um diagrama de estados UML permite representar, de forma rigorosa, quais os estados em que uma encomenda pode encontrar-se e quais as transições válidas entre eles, desencadeadas por eventos do sistema ou por ações explícitas dos utilizadores.

A Fig. 12 apresenta o diagrama de estados proposto para a Encomenda. Quando uma encomenda é criada, entra no estado Registada. A partir desse estado, se existir um veículo disponível com capacidade adequada, a encomenda transita para Atribuída; caso contrário, fica no estado Pendente, aguardando que um veículo se torne disponível. Uma vez atribuída, o início efetivo do transporte leva a encomenda ao estado Em Transporte e, quando a entrega é confirmada, ao estado final Entregue. A qualquer momento antes da entrega, o processo pode ser interrompido através da transição Cancelar, conduzindo a encomenda ao estado final Cancelada. Este modelo de estados suporta diretamente o requisito não funcional de fiabilidade (secção 4.2), ao garantir que as transições inválidas — por exemplo, passar diretamente de Registada a Entregue, sem transporte — são rejeitadas pela lógica do sistema.

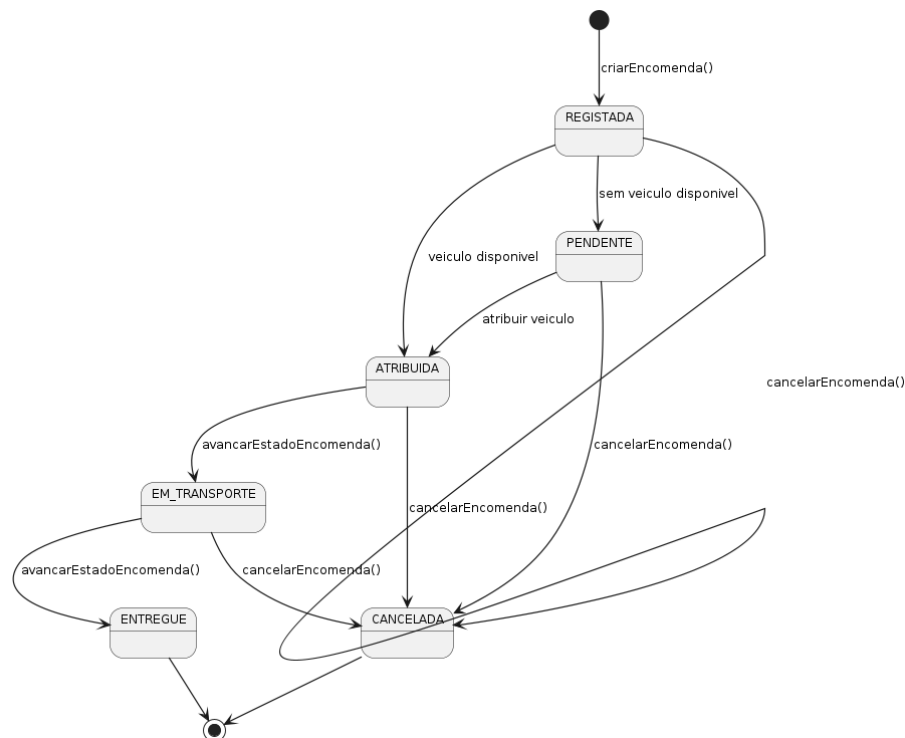


Fig. 12. Diagrama de estados da Encomenda, indicando as transições entre os estados Registada, Pendente, Atribuída, Em Transporte, Entregue e Cancelada.

2.5 Diagrama de Pacotes

O diagrama de pacotes (Fig. 13) apresenta a organização do sistema em agrupamentos lógicos de classes, promovendo modularidade, separação de responsabilidades e facilidade de manutenção. Na solução proposta são considerados cinco pacotes principais: um pacote de interface de **utilizador** (ui), responsável pelo menu de interação em consola; um pacote de **controlo** (controller), onde reside a classe Sistema; um pacote **repositório** (repository) para gerar Id, Procurar, Remove e Obter Informação geral; um pacote de **Serviço** (service) que tem parte da logica do código com recomendar Veículos; e um pacote de **domínio** (domain), que reúne as classes Cliente, Encomenda, Veículo e Depósito. Esta estrutura materializa, a nível arquitetural, o padrão Low Coupling discutido na secção anterior.

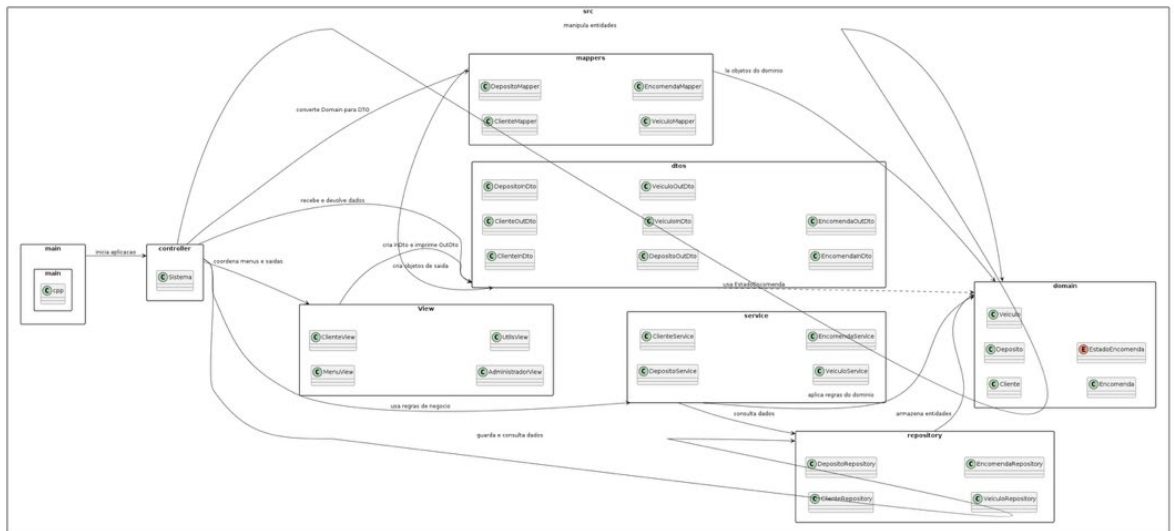


Fig. 13. Diagrama de pacotes do sistema.

2.6 Diagrama de componentes

O diagrama de componentes (Fig. 14) ilustra a arquitetura do sistema na forma de componentes funcionais que comunicam entre si. Desta perspetiva, evidencia-se a separação entre a interface do utilizador e os módulos centrais de gestão — gestão de encomendas e gestão de veículos — coordenados por um Sistema Central.

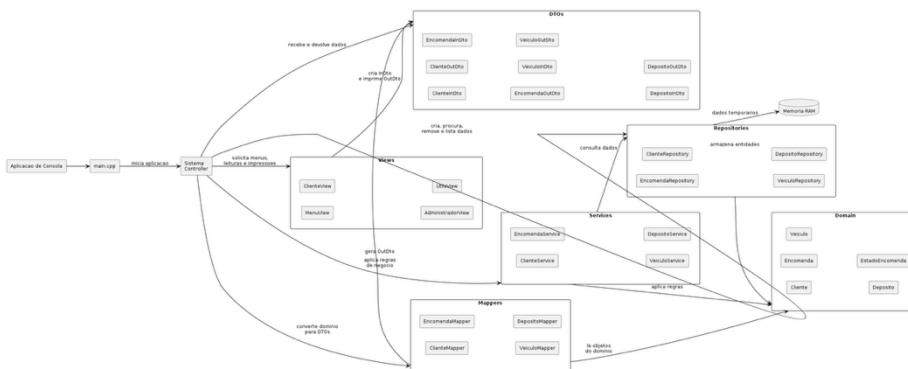


Fig. 14. Diagrama de Componentes

2.7 Diagrama de Instalação

Finalmente, o diagrama de instalação (deployment diagram) representa a distribuição física dos componentes de software pelos dispositivos de hardware onde serão executados. Uma vez que o sistema é uma aplicação local em C++, executada em modo de consola, o diagrama de instalação é relativamente simples, como se verifica na Fig. 15: a aplicação correrá integralmente no computador do utilizador, sem necessidade de infraestrutura de rede ou de servidores externos.

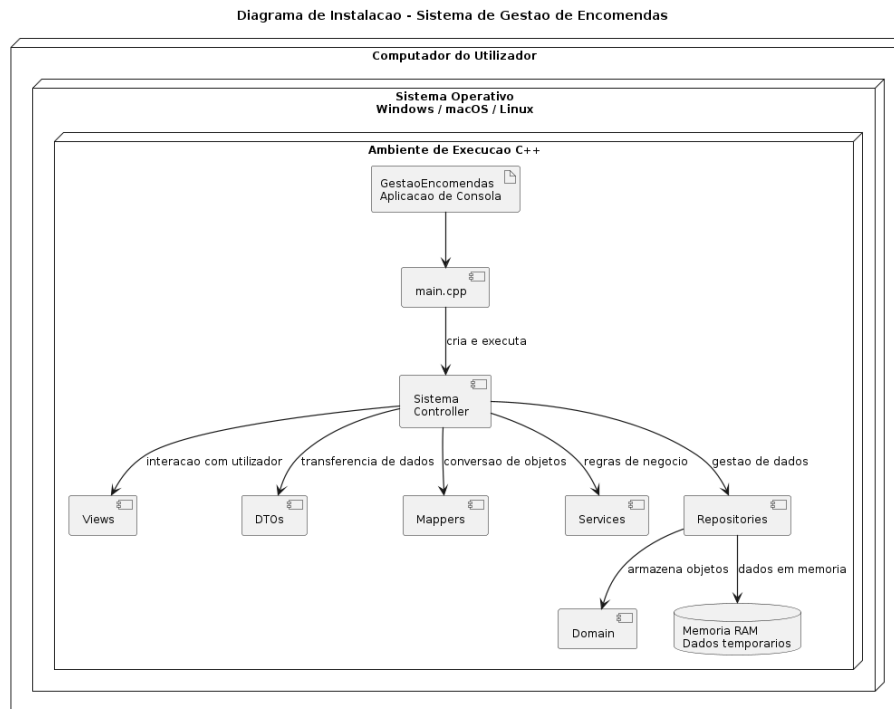


Fig 15. Diagrama de Instalação

3. Implementação

A implementação foi realizada em C++, com execução local em consola. O código foi organizado por responsabilidades, evitando concentrar toda a lógica no main.cpp ou nas Views. O main.cpp apenas cria o Sistema e chama run(), enquanto as interações com o utilizador são tratadas pelas classes View

3.1 Estrutura de Pastas

A estrutura de pastas deve ser apresentada através de uma figura ou listagem. A organização recomendada é a seguinte:

```

src/
  main.cpp
  controller/
    Sistema.h / Sistema.cpp
  View/
    headers/ e sources/
    MenuView, ClienteView, AdministradorView, UtilsView
  domain/
    Cliente, Encomenda, Veiculo, Deposito
  dtos/
    ClienteInDto, ClienteOutDto, EncomendaInDto, EncomendaOutDto,
    VeiculoInDto, VeiculoOutDto, DepositoInDto, DepositoOutDto
  mappers/
    ClienteMapper, EncomendaMapper, VeiculoMapper, DepositoMapper

```

```

repository/
  ClienteRepository, EncomendaRepository, VeiculoRepository, DepositoRepository
service/
  ClienteService, EncomendaService, VeiculoService, DepositoService
CMakeLists.txt

```

3.2 Arquitetura do Sistema

A aplicação segue uma arquitetura em camadas, com características semelhantes ao padrão Model-View-Controller (MVC), adaptada ao contexto de uma aplicação em C++ executada em consola. Nesta organização, o ficheiro `main.cpp` tem apenas a responsabilidade de iniciar o programa, criando um objeto da classe `Sistema` e chamando o método `run()`.

A camada de apresentação é composta pelas classes `MenuView`, `ClienteView`, `AdministradorView` e `UtilsView`. Estas classes são responsáveis por apresentar menus, recolher dados introduzidos pelo utilizador e mostrar mensagens ou resultados. Desta forma, a interação com o utilizador fica separada da lógica principal da aplicação.

A camada de controlo é representada pela classe `Sistema`, que funciona como controlador principal. Esta classe coordena o fluxo da aplicação, recebe os dados vindos das Views através de DTOs, chama os métodos necessários dos repositórios e serviços, e devolve os resultados às Views também através de DTOs. Assim, o controlador não imprime diretamente informação no ecrã, mantendo a separação entre lógica de controlo e interface.

A camada de domínio contém as classes fundamentais do sistema, nomeadamente `Cliente`, `Encomenda`, `Veiculo` e `Deposito`. Estas classes representam os principais conceitos do problema e armazenam os dados essenciais de cada entidade. Para evitar que as Views acedam diretamente às classes do domínio, foram introduzidos DTOs, como `ClienteInDto`, `ClienteOutDto`, `EncomendaInDto` e `EncomendaOutDto`, que servem para transportar dados entre as camadas.

Além disso, foram criadas classes `Mapper`, responsáveis por converter objetos do domínio em DTOs de saída. As classes `Repository`, como `ClienteRepository`, `EncomendaRepository`, `VeiculoRepository` e `DepositoRepository`, são responsáveis pelo armazenamento em memória, pesquisa, remoção e listagem dos objetos. Por fim, as classes `Service` concentram parte das regras de negócio e validações, contribuindo para uma melhor organização do código.

Esta arquitetura melhora a modularidade do sistema, reduz o acoplamento entre classes e facilita futuras alterações, como a introdução de persistência em ficheiros, testes automatizados ou novas funcionalidades.

3.3 Classes Principais

As Classes principais foram organizadas de acordo com a função que desempenham na aplicação. A tabela seguinte resume as responsabilidades mais importantes.

Tabela 11- Classes principais.

Grupo	Classes	Responsabilidade
Controller	<code>Sistema</code>	Coordena o fluxo principal, chama Views, Services, Repositories e Mappers.

Views	MenuView, ClienteView, AdministradorView, UtilsView	Mostram menus, leem dados, imprimem resultados e mensagens gerais.
Domain	Cliente, Encomenda, Veiculo, Deposito	Representam os conceitos centrais do problema.
DTOs	InDTOs e OutDTOs	Transportam dados de entrada e saída entre a interface e o sistema.
Mappers	ClienteMapper, EncomendaMapper, VeiculoMapper, DepositoMapper	Transformam objetos do domínio em DTOs de saída
Repositories	ClienteRepository, EncomendaRepository, VeiculoRepository, DepositoRepository	Armazenam objetos em std::vector e disponibilizam operações CRUD simples.
Services	ClienteService, EncomendaService, VeiculoService, DepositoService	Concentram validações e regras auxiliares de negócio

3.4 Tecnologias Utilizadas

Na realização do sistema usou-se algumas tecnologias como:

Tabela 12-Tecnologias Usadas

Tecnologia	Utilização
C++	Linguagem principal da aplicação
STL-std:vector	Armazenamento em memória das listas de clientes, depósitos, veículos e encomendas.
STL-std:string	Tratamento de texto, nomes, contactos, descrições e mensagens.
CMake	Configuração do processo de compilação
CLion	Ambiente de desenvolvimento utilizado.
UML	Modelação do domínio, classes e sequências
Consola	Interface de interação com o utilizador

4 Testes e Validação

A validação do sistema foi realizada através da implementação de testes unitários automatizados, recorrendo ao framework **Google Test (GoogleTest)** para C++.

Os testes foram estruturados segundo o padrão **Arrange – Act – Assert**, permitindo organizar cada teste em três fases distintas:

- **Arrange:** preparação dos objetos e dados de teste;
- **Act:** execução das operações a testar;
- **Assert:** verificação dos resultados esperados.

A estratégia adotada focou-se na validação das regras de negócio implementadas na camada de serviços e no controlador Sistema, garantindo o correto funcionamento dos principais fluxos da aplicação.

4.1 Organizacao dos Testes

Os testes foram organizados numa pasta dedicada (/ProjectTest), separada do código principal da aplicação, promovendo uma estrutura modular e de fácil manutenção.

A suite de testes foi dividida por componentes do sistema:

- ClienteServiceTest.cpp
- VeiculoServiceTest.cpp
- DepositoServiceTest.cpp
- EncomendaServiceTest.cpp
- SistemaControllerTest.cpp

Cada ficheiro agrupa testes relacionados com uma determinada funcionalidade, permitindo validar individualmente cada módulo do sistema. Os testes incidem principalmente sobre a classe Sistema, que atua como controlador, possibilitando validar indiretamente a interação entre **Services, Repositories e entidades de domínio**.

4.2 Casos de Teste Implementados

Foram implementados testes unitários para cobrir os principais cenários funcionais e regras de negócio do sistema.

Criação de Encomendas

Foi desenvolvido um teste que valida a criação de encomendas com dados válidos, garantindo que o sistema gera corretamente um identificador e atribui estado inicial adequado.

Validação de Dados

Foram incluídos testes que verificam a rejeição de dados inválidos, como pesos negativos ou campos obrigatórios não preenchidos.

Cancelamento de Encomendas

Foi testado o cancelamento de encomendas, garantindo que:

- apenas encomendas em estados válidos podem ser canceladas;
- o veículo associado é corretamente libertado;
- a capacidade do depósito é reposta.

Transição de Estados

Foram validadas as transições entre estados:

- ATRIBUIDA → EM_TRANSPORTE
- EM_TRANSPORTE → ENTREGUE

Garantindo o cumprimento da máquina de estados definida no sistema.

Atribuição Automática de Veículos

Um dos testes mais relevantes valida o comportamento do sistema quando uma encomenda pendente passa automaticamente para o estado **ATRIBUIDA**, após a liberação de um veículo.

Este teste assegura que o sistema reage corretamente à disponibilidade de recursos e reatribui veículos de forma dinâmica.

Gestão de Veículos

Foram implementados testes para:

- adicionar veículos;
- impedir duplicação de matrículas (regra de unicidade);
- impedir remoção de veículos em uso.

Testes de Construtores e Operadores

Foram também testados:

- construtores das classes de domínio (validação de dados obrigatórios);
- operadores de comparação (operator==), permitindo validar igualdade entre objetos.

4.3 Cobertura dos Testes

A suite de testes cobre os principais aspetos do sistema:

- Criação e validação de entidades
- Regras de negócio (ex.: unicidade de matrícula)
- Fluxos funcionais (criar, cancelar, avançar estado)
- Gestão de recursos (veículos e depósitos)
- Consistência entre estados (ex.: veículo ocupado vs. encomenda atribuída)

Desta forma, garante-se a validação das funcionalidades críticas e a robustez do sistema.

4.4 Execução dos Testes

Os testes são executados automaticamente no ambiente de desenvolvimento CLion, através do GoogleTest.

Cada teste produz um resultado de:

- **PASS** — comportamento esperado validado;
- **FAIL** — comportamento incorreto ou inconsistente.

Esta abordagem permite identificar rapidamente erros e validar alterações ao código.

4.5 Vantagens dos Testes Unitários

A introdução de testes unitários trouxe diversos benefícios ao sistema:

- Detecção precoce de erros;
- Maior confiança na evolução do código;
- Verificação automática de regras de negócio;
- Garantia de consistência entre componentes.

Adicionalmente, os testes permitiram identificar e corrigir problemas de consistência, como a sincronização entre o estado das encomendas e a disponibilidade dos veículos.

4.6 Relação entre Testes e Casos de Uso

Os testes unitários foram definidos com base nos casos de uso identificados no sistema, garantindo a validação prática dos requisitos funcionais.

A tabela seguinte apresenta a correspondência entre os testes implementados e os respectivos casos de uso:

Tabela 13-Test e UC

Caso de Uso	Descrição	Testes Associados(PASS)	Objetivo da Validação
Registrar Cliente	Permite adicionar clientes ao sistema	RegistrarCliente	Verificar criação correta e geração de ID
Remover Cliente	Permite remover clientes existentes	RemoverCliente, NaoRemoverClienteInexistente	Garantir remoção válida e tratar erros
Criar Encomenda	Regista uma nova encomenda	CriarEncomendaValida, CriarEncomendaPesoInvalido	Validar entradas e criação
Consultar Estado da Encomenda	Permite consultar estado	SistemaTest.CriarConsultar	Confirmar retorno correto da informação
Cancelar Encomenda	Cancela encomenda existente	CancelarEncomendaValida, CancelarEncomendaJaEntregue	Garantir regras de estado
Adicionar Veículo	Adiciona veículos ao sistema	CriarVeiculo	Validar criação
Remover Veículo	Remove veículos	RemoverVeiculo, NaoRemoverVeiculoEmUso	Bloquear remoção quando em uso
Consultar Informação Global	Lista dados do sistema	(implícito via get...)	Validar integridade dos dados apresentados

5 Glossário

A Tabela 13 reúne os termos técnicos utilizados ao longo deste relatório, acompanhados das respectivas definições, de modo a facilitar a leitura do documento por parte de leitores com formações distintas.

Tabela 13-Glossario

Termo	Definição
Administrador	Utilizador responsável pela gestão geral do sistema, incluindo veículos, clientes, depósitos e consulta de informação global.
Cliente	Utilizador que se regista no sistema, cria encomendas e consulta o estado das suas entregas.
Encomenda	Pedido de transporte criado por um cliente, contendo descrição, peso, depósito de origem, depósito de destino, veículo associado e estado.
Veículo	Meio de transporte utilizado para transportar encomendas entre depósitos. Possui matrícula, capacidade máxima e estado de disponibilidade.
Depósito	Ponto físico de recolha ou entrega de encomendas. Cada depósito possui nome, localização e capacidade máxima.
Estado da Encomenda	Situação atual de uma encomenda no sistema, podendo ser registada, pendente, atribuída, em transporte, entregue ou cancelada.
UML	Linguagem de modelação usada para representar visualmente a estrutura e o comportamento do sistema através de diagramas.
Caso de Uso	Descrição de uma funcionalidade do sistema do ponto de vista do utilizador.
User Story	Descrição curta de uma necessidade do utilizador, normalmente escrita na forma “Como utilizador, quero fazer algo para obter um benefício”.
Requisito Funcional	Funcionalidade que o sistema deve realizar, como registar cliente, criar encomenda ou adicionar veículo.
Requisito Não Funcional	Característica de qualidade do sistema, como usabilidade, desempenho, fiabilidade e facilidade de manutenção.
FURPS+	Modelo usado para organizar requisitos não funcionais, incluindo funcionalidade, usabilidade, fiabilidade, desempenho, suportabilidade e restrições.
Matriz de Rastreabilidade	Tabela que relaciona user stories, requisitos e casos de uso, garantindo que cada necessidade do sistema está representada no desenho.
MVC	Padrão arquitetural que separa a aplicação em camadas de apresentação, controlo e modelo. No projeto, esta ideia foi adaptada à aplicação em consola.
View	Camada responsável pela interação com o utilizador, incluindo menus, leitura de dados e apresentação de resultados.
Controller	Camada responsável por coordenar o fluxo da aplicação. No projeto, esta responsabilidade é assumida pela classe Sistema.

Domain	Camada que contém as entidades principais do problema, como Cliente, Encomenda, Veiculo e Depósito.
DTO	Objeto de transferência de dados usado para transportar informação entre as Views e o Sistema, sem expor diretamente as classes do domínio.
InDto	DTO usado para transportar dados de entrada introduzidos pelo utilizador.
OutDto	DTO usado para transportar dados de saída que serão apresentados ao utilizador.
Mapper	Classe responsável por converter objetos do domínio em DTOs de saída.
Repository	Classe responsável por armazenar, procurar, listar e remover objetos em memória.
Service	Classe responsável por concentrar regras de negócio e validações do sistema.
GRASP	Conjunto de princípios usados para atribuir responsabilidades às classes de forma organizada e com baixo acoplamento.
Controller GRASP	Princípio aplicado à classe Sistema, que recebe e coordena os pedidos principais da aplicação.
Low Coupling	Princípio que procura reduzir dependências entre classes, facilitando alterações futuras.
High Cohesion	Princípio que defende que cada classe deve ter responsabilidades bem definidas e relacionadas entre si.
Information Expert	Princípio que atribui uma responsabilidade à classe que possui a informação necessária para a executar.
Pure Fabrication	Classe criada por razões técnicas de organização, mesmo que não represente diretamente um conceito do mundo real.
Persistência	Capacidade de guardar dados de forma permanente. No estado atual do projeto, os dados são guardados apenas em memória durante a execução.
std::vector	Estrutura da biblioteca padrão de C++ usada para armazenar listas dinâmicas de objetos, como clientes, veículos e encomendas.
std::string	Tipo da biblioteca padrão de C++ usado para representar texto.
Aplicação de Consola	Programa executado através de texto no terminal, sem interface gráfica.
Testes Unitario	Um teste que verifica se uma parte pequena do código, como uma função ou método, funciona corretamente. Neste projeto usou-se como base o GoogleTest Unit c++.

6 Conclusão

O trabalho desenvolvido no âmbito deste relatório técnico permitiu conceber e implementar uma solução orientada a objetos para a gestão de encomendas numa pequena cidade com vários pontos de distribuição. Ao longo do projeto foram aplicadas diferentes etapas da Engenharia de Software, desde a identificação de requisitos funcionais e não funcionais, à definição de user stories, modelação do domínio, elaboração de diagramas UML e implementação da aplicação em C++.

Um dos aspetos mais relevantes do projeto foi a evolução da arquitetura interna do sistema. Inicialmente, a aplicação apresentava uma estrutura mais simples, mas, com o desenvolvimento do código, tornou-se necessário reorganizar responsabilidades e separar melhor as diferentes camadas. Assim, a solução passou a seguir uma arquitetura em camadas, com características próximas do padrão Model-View-Controller. A camada de apresentação ficou representada pelas classes MenuView, ClienteView, AdministradorView e UtilsView, responsáveis pela interação com o utilizador. A classe Sistema passou a atuar como controlador principal, coordenando o fluxo da aplicação e intermediando a comunicação entre as Views, os DTOs, os Repositories, os Services e as classes do domínio.

A introdução de DTOs permitiu separar os dados de entrada e saída das entidades do domínio, evitando que as Views acessem diretamente a classes como Cliente, Encomenda, Veiculo e Deposito. Além disso, a utilização de Mappers tornou possível converter objetos do domínio em objetos de saída, mantendo uma comunicação mais organizada entre as camadas. Os Repositories ficaram responsáveis pelo armazenamento em memória, pesquisa, remoção, listagem e geração de identificadores, enquanto os Services passaram a concentrar parte das regras de negócio e validações do sistema.

Esta reorganização contribuiu para uma solução mais modular, clara e fácil de manter, aplicando princípios GRASP como Controller, Low Coupling, High Cohesion, Information Expert e Pure Fabrication. A separação entre interface, controlo, domínio, serviços, repositórios, DTOs e mappers tornou o código mais compreensível e preparado para futuras alterações.

Durante a implementação foram desenvolvidas funcionalidades essenciais, como o registo de clientes, criação de encomendas, seleção de depósitos de origem e destino, atribuição de veículos, consulta do estado de encomendas, cancelamento de encomendas, adição e remoção de veículos, remoção de clientes e consulta de informação global pelo administrador. Estas funcionalidades demonstram que o sistema cumpre os principais objetivos definidos na fase de análise.

Apesar dos resultados alcançados, o sistema ainda apresenta possibilidades de evolução. Atualmente, os dados são mantidos apenas em memória durante a execução da aplicação, pelo que uma melhoria futura seria a introdução de persistência em ficheiros ou base de dados. Também seria importante desenvolver testes automatizados, melhorar as validações, adicionar uma interface gráfica e introduzir funcionalidades complementares, como cálculo automático de custos de transporte, notificações ao cliente e relatórios administrativos.

Conclui-se que o projeto permitiu consolidar conhecimentos de Programação Orientada a Objetos, arquitetura em camadas, UML, padrões GRASP e organização de projetos em C++. A solução desenvolvida demonstra como um sistema de gestão de encomendas pode ser estruturado de forma modular, organizada e evolutiva, mantendo uma separação clara entre as responsabilidades de cada componente.

7 Referências Bibliográficas

1. Larman, C.: Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development. 3rd edn. Prentice Hall, Upper Saddle River (2005)
2. Sommerville, I.: Software Engineering. 10th edn. Pearson Education, Harlow (2016)
3. Pressman, R.S., Maxim, B.R.: Software Engineering: A Practitioner's Approach. 8th edn. McGraw-Hill Education, New York (2014)
4. Object Management Group: Unified Modeling Language (UML) Specification, version 2.5.1. OMG Document formal/2017-12-05 (2017), <https://www.omg.org/spec/UML>
5. IEEE: IEEE Recommended Practice for Software Requirements Specifications. IEEE Std 830-1998. IEEE, New York (1998)
6. Sousa, P.B., Freitas, C.: Fundamentos de Desenvolvimento de Software — Material da Unidade Curricular. Instituto Superior de Engenharia do Porto (2025)
7. Grady, R.B.: Practical Software Metrics for Project Management and Process Improvement. Prentice Hall, Englewood Cliffs (1992)